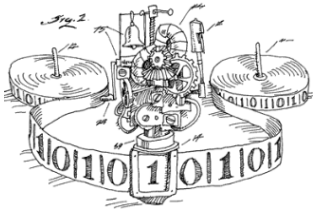
Teoría de la Computación y Lenguajes Formales



Teoría de la Computación y Lenguajes Formales: Sesión 2. Tipos de Cálculo

Édgar Chacón Ramírez
Departamento de Computación
Semestre 2026 - B

Alcance de la sesión 2

UNIVERSIDAD DE LOS ANDES VENEZUELA

Introducción
- Cálculo combinatorio
- Cálculo secuencial

Máquinas que simulan máquinas

Flujos de Trabajo

UNIVERSIDAD DE LOS ANDES VENEZUELA

La Gran Dicotomía

Existen dos formas de procesar la información:

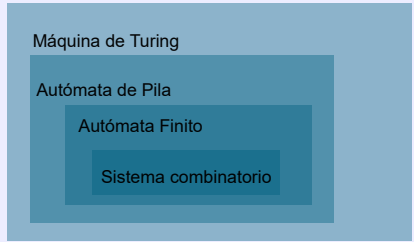
- Sin memoria: El resultado es una foto fija de la entrada actual.

$$y = f(x)$$

- Con memoria: El resultado es una película que depende de todo lo que ha pasado antes.

$$\begin{aligned} y_k &= f_1(x_k, estado_k) \\ estado_{k+1} &= f_2(x_k, estado_k) \end{aligned}$$

La máquina que procesa el cálculo asocia las entradas y un valor almacenado en memoria.



Definición

- La salida depende **ÚNICA Y EXCLUSIVAMENTE** de las entradas actuales.
- **Ecuación:** $Salida(t) = f(Entrada(t))$
- **Propiedad fundamental:** Es puro. Ante los mismos estímulos, siempre reacciona igual.
- **Comportamiento:** No tiene historia. El tiempo no importa (salvo el retardo físico de propagación)

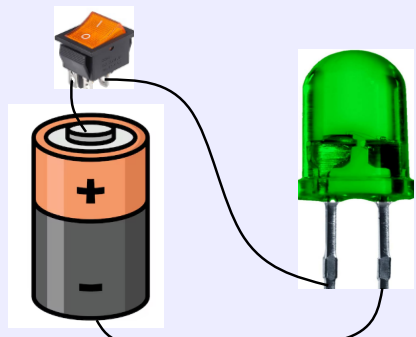
Ejemplo Una calculadora de bolsillo que hace sumas. Si pones $2 + 2$, siempre devuelve 4, sin importar cuántas operaciones hayas hecho antes.

Sistema combinatorio. Comportamiento

UNIVERSIDAD DE LOS ANDES VENEZUELA

Comportamiento

El comportamiento siempre será el mismo.
El led se enciende cuando el interruptor deja pasar la corriente.



Cálculo combinatorio. Implementación en hardware

Circuitos Lógicos

Cuando la decisión no depende de la historia, basta con combinar compuertas: AND, OR, NOT, XOR conectadas sin realimentación (sin lazos hacia atrás).

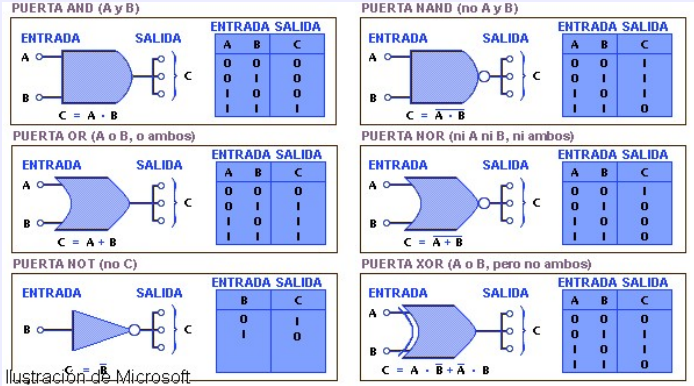


Ilustración de Microsoft

Propiedad clave: el circuito no tiene elementos de memoria (flip-flops, latches). La salida cambia únicamente cuando cambia la entrada.

Programa de computación

El equivalente en código es una función pura: sin variables globales, sin estado interno entre llamadas.

Ejemplo

$$aprobar_pedido(cantidad, umbral) \;=\; cantidad < umbral$$

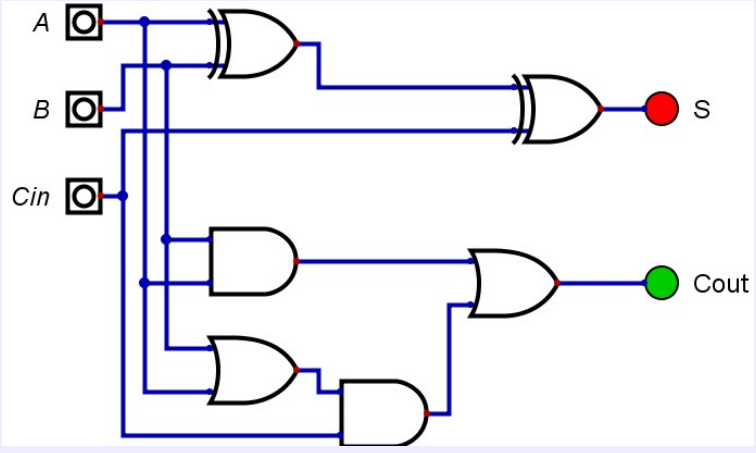
Misma entrada, misma salida. No importa cuántas veces se haya llamado antes.

Propiedad clave: no hay diferencia relevante entre implementarlo en una compuerta física o en una línea de código — es el mismo cálculo combinatorio, solo cambia el sustrato.

Cálculo combinatorio. Implementación

Sumador binario

Ejemplo: Combinación de compuertas lógicas para tener un sumador de un bit



Cálculo combinatorio. Implementación

Una aplicación con decisiones sin memoria, será combinatoria

Ejemplo1.py > ...

```python
# Ejemplo1.py
a = int (input())
if a > 5:
    print("El numero ingresado es mayor a 5")
else:
    print("El numero ingresado es menor a 5")

```

Ejemplo en procesos de negocios

Un sistema que lea la cantidad de productos en almacén y realice un pedido si la cantidad de producto es inferior a una cierta cantidad.

El Cálculo Secuencial (Basado en Historia / Estado)

UNIVERSIDAD DE LOS ANDES VENEZUELA

Definición

La salida depende de las entradas actuales Y del estado interno del sistema (que resume toda la historia previa).

$$\begin{aligned} y_k &= f_1(x_k, estado_k) \\ estado_{k+1} &= f_2(x_k, estado_k) \end{aligned}$$

Ejemplo en un proceso de negocio

Hacer una compra dependiendo del nivel del inventario. Se realiza si no se hay un pedido pendiente por llegar.

Máquina expendedora

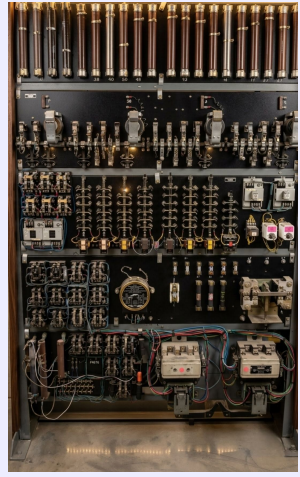
Entrega el producto luego de introducir la moneda.

$$Moneda \rightarrow Producto$$

El Cálculo Secuencial (Basado en Historia / Estado): Implementación

Ejemplo de implantación

Circuito a relés de un controlador de ascensor.



La construcción de una solución específica es demasiado costosa.

Máquinas que simulan máquinas

Implantación de soluciones usando máquinas de Gramática 0

UNIVERSIDAD DE LOS ANDES VENEZUELA

Resultado formal: Computador ≡ reconocedor de Gramáticas Tipo 0

La equivalencia Chomsky–Turing es un teorema: las gramáticas de Tipo 0 generan exactamente los lenguajes recursivamente enumerables, y ese es precisamente el conjunto que una máquina de Turing puede reconocer.

Matiz importante: "reconocer" aquí significa *semidecidir*. La máquina confirma cuando una cadena pertenece al lenguaje, pero puede no detenerse si no pertenece. Ese es el precio de la generalidad de Tipo 0.

Tesis de modelado (no es un teorema, es nuestra propuesta de curso)

Si un computador reconoce Tipo 0, entonces procesos reales (físicos, biológicos, de negocio) —incluyendo su parte continua, si la tienen— pueden describirse mediante un lenguaje formal y validarse con esa descripción.

Esto **no se deriva automáticamente** del teorema: depende de cómo discreticemos/muestreemos el proceso. Es la apuesta de modelado que sostiene este curso, no un hecho matemático.

De la Gramática a la Máquina

UNIVERSIDAD DE LOS ANDES VENEZUELA

Dos objetos, una misma idea

- La gramática describe **qué** cadenas son válidas (la especificación del proceso).
- La máquina (autómata) decide **si** una cadena dada es válida, ejecutando la especificación paso a paso.

Idea central de la sesión: si describimos un proceso de negocio como gramática, cualquier máquina que reconozca esa clase de gramática puede **validar** y **ejecutar** el proceso, sin programarlo a mano.

Formalizando el ejemplo de la máquina expendedora

UNIVERSIDAD DE LOS ANDES VENEZUELA

Retomando el ejemplo secuencial

$$\textit{Moneda} \rightarrow \textit{Producto}$$

Como autómata de estados, la máquina expendedora es:

- Estados: {Esperando, Pagado, Entregando}
- Alfabeto de entrada: {moneda, selección, retiro}
- Transiciones: Esperando $\xrightarrow{\text{moneda}}$ Pagado $\xrightarrow{\text{selección}}$ Entregando $\xrightarrow{\text{retiro}}$ Esperando

Esto **ya es** una gramática (de hecho, Tipo 3 / regular): cada transición es una regla de producción.

Dos caminos para implementar la máquina

UNIVERSIDAD DE LOS ANDES VENEZUELA

Interpretar	Compilar
Un motor genérico lee la descripción formal (la gramática/autómata) y la ejecuta en tiempo real.	La descripción formal se traduce a código específico para el computador, una sola vez.
Ventaja: cambiar el proceso es cambiar la descripción, no el código.	**Ventaja:** ejecución más rápida, sin el costo del motor genérico.

En FÉNIX

El modelo DIS-MOL-DIL se describe formalmente y un motor lo **interpreta** en tiempo de ejecución: el mismo motor sirve para distintas plantas, solo cambia la descripción del proceso.

Tres problemas, tres gramáticas

UNIVERSIDAD DE LOS ANDES VENEZUELA

La jerarquía en acción (Ejemplos introductorios)

Cajero (Tipo 3)	**Compra / Ensamble (Tipo 2)**	**Garantía cruzada (Tipo 1)**
Secuencia lineal: *Insertar* $\rightarrow$ *PIN* $\rightarrow$ *Retirar*	Anidamiento de subprocesos: *Coche* $\rightarrow$ *Motor* $\rightarrow$ *Bloque* $\rightarrow$ *Pistones*	Dependencia entre partes distantes: El número de serie declarado en la **orden** debe coincidir con el grabado en la **pieza entregada**
Sin memoria auxiliar	Necesita Pila (sub-llamadas anidadas)	Necesita cinta acotada (comparar dos zonas separadas de la misma cadena)
Regla de oro: la máquina más compleja siempre puede **simular** a las anteriores.		

El salto

El orquestador que construiremos será un intérprete (Tipo 0) capaz de leer y ejecutar **cualquiera** de estas descripciones.

Flujos de Trabajo

Flujos de Trabajo

UNIVERSIDAD DE LOS ANDES VENEZUELA

Definición

Un Flujo de Trabajo (workflow) es una secuencia ordenada de pasos, tareas o actividades repetitivas que se realizan en un orden determinado para completar un proceso y alcanzar un objetivo específico.
Funciona como un mapa o ruta que define qué se hace, quién lo hace y en qué orden.
Elementos principales:

- Reglas del Proceso
- Recursos
- Producto

Flujos de Trabajo: Relación con la jerarquía de gramáticas

UNIVERSIDAD DE LOS ANDES VENEZUELA

A mayor complejidad del flujo, mayor poder de la gramática

- Secuencia ordenada de pasos $\mapsto$ Autómata de Estado Finito (Tipo 3)
- Paso compuesto de sub-pasos (anidamiento) $\mapsto$ Autómata a Pilas (Tipo 2)
- Evolución dependiente de otros elementos (sensible al contexto) $\mapsto$ Autómata Linealmente Acotado (Tipo 1)

Ejemplo: flujo de aprobación de pedidos

- Recibir $\rightarrow$ Verificar $\rightarrow$ Aprobar: secuencia simple, Tipo 3.
- Si "Verificar" incluye sub-pasos (validar stock, validar crédito, y solo entonces continuar): anidamiento, Tipo 2.
- Si la aprobación depende del historial de pedidos previos del mismo cliente: sensible al contexto, Tipo 1.

Flujos de Trabajo: cuando la jerarquía no alcanza

UNIVERSIDAD DE LOS ANDES VENEZUELA

Las Redes de Petri no son un nivel más de la jerarquía

La jerarquía de Chomsky describe grados de restricción sobre secuencias. Pero un workflow real casi nunca es una única secuencia: tiene actividades que ocurren **en paralelo**.

- Ningún nivel de la jerarquía (ni siquiera Tipo 0) modela concurrencia de forma natural: todos describen, en el fondo, *una* cadena que se genera o reconoce paso a paso.
- Las Redes de Petri son un formalismo distinto, diseñado específicamente para paralelismo, sincronización y recursos compartidos.

Mismo ejemplo, ahora con paralelismo

Validar stock y validar crédito pueden ocurrir **al mismo tiempo**, y la aprobación espera a que **ambas** terminen. Ninguna gramática de la jerarquía captura esto de forma natural — es el terreno de las Redes de Petri.

Flujos de Trabajo: Gramática vs. Ejecución Concurrente

UNIVERSIDAD DE LOS ANDES VENEZUELA

La gramática describe una cadena; el paralelismo no es una cadena

Una gramática (de cualquier tipo, incluyendo Tipo 0) genera o reconoce una secuencia de símbolos, uno detrás del otro. Eso es cierto incluso si el símbolo se llama "PAR": sigue siendo un símbolo más en una cadena lineal.

- Anotar "esto es paralelo" en la especificación **no le da** a la gramática el poder de expresar paralelismo real — es una etiqueta que un intérprete externo decide interpretar de forma especial.
- El paralelismo real (dos ramas avanzando *a la vez*, sincronizándose después) requiere un modelo distinto: eso es exactamente lo que las Redes de Petri formalizan de manera nativa.

Cómo lo resolveremos en la práctica (el orquestador)

1. La especificación usa fork/join como anotación (no como regla gramatical):

$$\text{Verificar} \longrightarrow \big[\text{Validar stock} \parallel \text{Validar crédito}\big] \longrightarrow \text{Aprobar}$$

2. El **motor** (no la gramática) lanza hilos en el fork y espera a que ambos terminen en el join, usando semáforos.

Conclusión: la gramática sigue siendo solo el *lenguaje de entrada* (secuencial); quien le da semántica

Flujos de Trabajo: cerrando el círculo

UNIVERSIDAD DE LOS ANDES VENEZUELA

¿Cómo se ejecuta un flujo descrito formalmente?

Ya sea que el flujo caiga en Tipo 3, Tipo 2, Tipo 1 o requiera Redes de Petri, queda la misma pregunta de la sesión anterior:

- Interpretar: un motor genérico lee la descripción del flujo y la ejecuta en tiempo real. Cambiar el proceso es cambiar la descripción.
- Compilar: la descripción se traduce a código específico una sola vez. Ejecución más rápida, menos flexible ante cambios.

En FÉNIX

El modelo DIS-MOL-DIL se **interpreta**: el mismo motor sirve para distintas plantas, solo cambia la descripción del proceso — ya sea que esa descripción use gramáticas de la jerarquía o Redes de Petri.

Cierre de la sesión 2

UNIVERSIDAD DE LOS ANDES VENEZUELA

Lo que vimos hoy

- La dicotomía combinacional / secuencial: con o sin memoria.
- Por qué construir cada máquina de estados a mano es costoso.
- El computador como reconocedor de Gramáticas Tipo 0 (teorema) y la tesis de que procesos reales pueden modelarse como lenguajes (nuestra apuesta de curso).
- Dos caminos de implementación: interpretar vs. compilar.
- Flujos de Trabajo y sus relaciones con las Gramáticas de Chomsky y las Redes de Petri

Próxima sesión

Las gramáticas describen *secuencias*. Pero muchos procesos de negocio tienen actividades concurrentes (en paralelo). Por eso necesitamos **Redes de Petri**. Intérprete + Motor

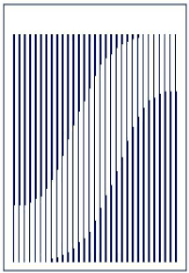
Conclusión final

Si no hay memoria, solo hay lógica. Si hay memoria, hay historia, y con historia aparece el tiempo y la sincronía

¡Muchas gracias!



UNIVERSIDAD
DE LOS ANDES
VENEZUELA



Édgar Chacón Ramírez
email: echacon@ula.ve